

TECHNICAL SESSION

• Modern API Architectures

Protocol Buffers & gRPC with Spring Boot

A developer's guide to high-performance microservices, binary serialization, and bidirectional streaming communication patterns.

PRESENTER DETAILS

Presentation by – Subhadip Layek, July 2026

SECTION 01

Protocol Buffers Core

Google's language-neutral, platform-neutral binary serialization mechanism.

What are Protocol Buffers?



Core Mechanism

Developed by Google for compact, language-neutral serialization of structured datasets. Perfect for modern distributed ecosystems.



Strict Contract-First Design

API structures are defined explicitly using highly type-safe `.proto` schemas, serving as a unified source of truth.



Binary Efficiency

Employs a dense, machine-optimized binary format. Eliminates massive string-parsing CPU tax associated with JSON/XML engines.



Polyglot Ecosystem

Compile once and target multiple languages. Automatically generates native client/server boilerplate across Java, Python, Go, and more.

Why Protocol Buffers?

Feature / Metric	JSON	XML	Protocol Buffers
Payload Format	Readable Plaintext Strings	Structured Plaintext Strings	Compact Machine-Binary
Payload Size	Standard / Moderate (Key inflation)	Extremely Heavy (Tags inflation)	Highly Compact (Minimal Bytes)
Processing Speed	Moderate Parsing Overhead	Slow & Verbose CPU-intensive parsing	Ultra-Fast Native CPU Binding
Schema Architecture	Optional / Untyped	Optional (XSD validation)	Strictly Required & Enforced
Inter-Language Support	Universal	Universal	Automatic via Stubs Compiler

💡 **Performance Gain:** Protocol Buffers are systematically **3-10x smaller** and **20-100x faster** than standard JSON/XML formats during serialization and network transport.

Proto3 Features & Capabilities

Supported DataTypes

Proto3 natively supports highly typed primitives without runtime cast issues:

double

float

int32

int64

bool

string

bytes

enum

map

Enums & Maps

```
enum Status { ACTIVE = 0; INACTIVE = 1; }

message User {
  Status status = 4;
  map<string, string> attributes = 5;
}
```

Nested Messages & Oneof

```
message Address {
  string street = 1;
  string city = 2;
}

message Contact {
  oneof contact_method {
    string email = 1;
    string phone = 2;
  }
}
```


Defining a .proto Schema

</> Basic .proto Definition

```
syntax = "proto3";

package com.digiwizkid.proto;

option java_multiple_files = true;
option java_package = "com.digiwizkid.protobuf";

message User {
    int32 id = 1;        // Field number tags
    string name = 2;
    string email = 3;
}
```

⚙️ Build Config & Code Gen

Compile target automatically compiles schema patterns into target Java files.

```
// build.gradle plugin task declaration
protobuf {
    protoc {
        artifact = 'com.google.protobuf:protoc:4.26.1'
    }
}
```

Build Output: Generates highly optimized immutable builders, `toByteArray()`, and `parseFrom()` utility helper stubs automatically.

The Protocol Buffers Workflow

A standard pipeline from service contract configuration to rapid runtime serialization.

1

Define Schema

Express strict data contracts within language-neutral .proto configuration templates.

2

Invoke Build

Gradle/Maven invokes Google's core native protoc runtime compiler utility.

3

Code Gen

Java objects, structures, and mutable Builders are safely compiled to the target folder.

4

Serialization

Dense, highly optimized binary streams run across microservice bounds instantly.

Serialization & Deserialization

[→ Serialization (Object → Bytes)

```
// Constructing immutable object via Builder
User user = User.newBuilder()
    .setId(101)
    .setName("Alice")
    .setEmail("alice@digiwizkid.com")
    .build();

// Convert to compact binary array
byte[] serializedData = user.toByteArray();
```

Payload size: 48 bytes vs equivalent JSON (~80+ bytes) containing verbose plain-text keys.

→] Deserialization (Bytes → Object)

```
// Instantly recreate raw object from bytes
User deserializedUser = User.parseFrom(serializedData);

System.out.println("Name: " + deserializedUser.getName());
System.out.println("Email: " + deserializedUser.getEmail());
```

Output: Name: Alice | Email: alice@digiwizkid.com. The round-trip perfectly preserves all data.

SECTION 02

gRPC with Spring Boot

Bringing high-performance Google Remote Procedure Calls to enterprise Java microservices.

What is gRPC?



Transport Speed

Next-generation remote procedure call framework. Leverages the robust power of HTTP/2 for bidirectional connection pathways.



Multiplexed Connections

Sends multiple concurrent bi-directional streams and requests over a single persistent, long-lived TCP connection.



Strict Binary Encoding

Driven purely by structured Protocol Buffer schemas instead of heavy, unvalidated, verbose text JSON bodies.



Native Streaming Capacity

Designed from scratch with built-in support for Client-Streaming, Server-Streaming, and Bidirectional pipelines.

gRPC vs REST API

Feature	Traditional REST API	Modern gRPC API
Transport Protocol	HTTP/1.1 (Blocking / Head-of-Line Bottlenecks)	HTTP/2 (Multiplexed bidirectional pipelines)
Payload Format	JSON / XML (Plaintext Strings)	Protobuf (Strict Machine Binary)
API Design Model	Resource-based (CRUD paths & endpoints)	Procedure-based (Direct RPC methods)
Streaming Capacities	Unidirectional (Requires Server-Sent Events)	Native client, server, & bidi streaming
Contract System	OpenAPI / Swagger (Optional specification)	Protocol Buffer schema (Strictly enforced)
Code Generation	Requires external generators & engines	Automatic from schema compile target

The 4 gRPC Communication Patterns



Unary RPC

Traditional 1-to-1 connection. The client fires a single target request and receives a single structured return payload.



Server Streaming

One request, multiple responses. Useful for continuous pipelines, active notifications, dashboard metrics, or file blocks.



Client & Bidi Stream

Streamed payloads from client, server, or both in real-time. Ideal for high-density IoT telemetry and chat networks.

Unary & Server Streaming

Unary Server Handler

```
@Override
public void unary(Message req,
    StreamObserver responseObserver) {

    Message reply = Message.newBuilder()
        .setText("Hello, " + req.getText())
        .build();

    responseObserver.onNext(reply);
    responseObserver.onCompleted();
}
```

Server Streaming Handler

```
@Override
public void serverStream(Message req,
    StreamObserver responseObserver) {

    for (int i = 1; i ≤ 5; i++) {
        responseObserver.onNext(Message.newBuilder()
            .setText("Chunk " + i + ": " + req.getText())
            .build());
        Thread.sleep(500);
    }
    responseObserver.onCompleted();
}
```


Client & Bidirectional Streaming

↓ Client Streaming

```
@Override
public StreamObserver clientStream(
    StreamObserver responseObserver) {
    return new StreamObserver<>() {
        int count = 0;
        public void onNext(Message m) { count++; }
        public void onCompleted() {
            responseObserver.onNext(Message.newBuilder()
                .setText("Total: " + count).build());
            responseObserver.onCompleted();
        }
    };
}
```

↔ Bidirectional Streaming

```
@Override
public StreamObserver bidiStream(
    StreamObserver responseObserver) {
    return new StreamObserver<>() {
        int count = 0;
        public void onNext(Message msg) {
            count++;
            responseObserver.onNext(Message.newBuilder()
                .setText("Echo: " + msg.getText()).build());
        }
        public void onCompleted() {
            responseObserver.onCompleted();
        }
    };
}
```


Spring Boot Integration

⚙️ application.yml Configuration

```
grpc:
  server:
    port: 9090          # gRPC bind port
  client:
    hello-service:
      address: 'static://localhost:9090'
      negotiationType: plaintext
```

The library `grpc-spring-boot-starter` handles autoconfiguration of Netty servers and client channels out of the box.

🔗 Native Service Annotations

```
// Annotate implementation classes
@GrpcService
public class GrpcStreamingDemoServiceImpl
    extends GrpcStreamingServiceGrpc.GrpcStreamingServiceImplBase {
    // Overrides ...
}

// Seamless client injection auto-wires stubs
@GrpcClient("hello-service")
private GrpcStreamingServiceGrpc.GrpcStreamingServiceBlockingStub stub;
```


Summary & Key Takeaways



Maximize Performance

Deploy Protocol Buffers to reduce computing bandwidth. Experience up to 10x compression over JSON datasets.



Native Streaming

Achieve bidirectional data flows without manual long-polling, complex server-sent events, or custom WebSocket setups.



Contracts First

Build robust interfaces. Prevent runtime type-mismatch faults across microservices using highly static `.proto` contracts.

 **Production Recommendation:** Choose gRPC/Protobuf for internal inter-service cluster communication (microservices), and keep traditional REST APIs only at public-facing border networks.

Industry Adoption

Proven at massive scale by the world's leading technology and streaming platforms.



Google

Internal microservice backbone



Netflix

Studio microservices orchestrator



Uber

High-throughput dispatch engine



Spotify

Inter-service telemetry & streaming



Docker

Daemon-to-client communication



Square

High-security transaction pipelines



Questions & Answers

Thank you for your attendance! Please feel free to raise any questions regarding binary serialization or HTTP/2 transport patterns.

Useful Links & Resources

- ✓ gRPC Official Documentation: <https://grpc.io>
- ✓ Protocol Buffers Documentation: <https://protobuf.dev>